Episode 8: AI Agent Meets Network Automation

Video Title

"Network as Code: An AI That Actually Understands Your Network"

Target Length

22-28 minutes

Goal

Show the AI assistant answering real questions about the live fabric using local open-source models. Demonstrate the MCP server, model selection, and how the structured data from Labs 1-6 makes AI integration possible. This is the finale. End strong.

Pre-Recording Checklist

Before you hit record, make sure:
- ContainerLab topology is running with clean deployed configs
- Ollama is running (either on the VM or Mac Mini)
- At least 2-3 models available (varying sizes)
- MCP server is working
- AI assistant can reach devices and answer queries
- Prepare 3-4 questions to ask (mix of easy and complex)

SEGMENT 1: Why AI + Network Automation (0:00 - 3:00)

What to show: Nothing on screen. This is a framing segment.

Talking points:
- "We have spent six labs building a complete Network as Code stack. Data model, validation, config generation, CI/CD, testing, drift detection. Every piece is automated, tested, and version-controlled."
- "But here is the thing about all that automation: it produces a lot of structured data. YAML data models, generated configs, test results, drift reports, routing tables, BGP session states. A human can read any one of these. But correlating them across six devices in real time takes effort."
- "Lab 7 connects an AI agent to the entire stack. It can answer questions like 'is the fabric healthy,' 'which devices are running as route reflectors,' or 'what changed since the last deployment.' And it answers them using live data, not cached summaries."
- "The key difference from a generic AI chatbot: this agent has tools. It can SSH into devices, read the data model, run validation, check drift. It is not hallucinating answers. It is pulling real data and interpreting it."

SEGMENT 2: The MCP Server (3:00 - 6:00)

What to show: Mention the MCP server concept without showing the code.

Talking points:
- "The integration point is an MCP server. MCP stands for Model Context Protocol. It is a standard way to give AI models access to tools."
- "Our MCP server exposes every tool we built across the seven labs. The validation engine, the config renderer, the deployment script, the test runner, the drift detector. Each one becomes a tool the AI can call."
- "When the AI needs to answer a question about BGP sessions, it does not guess. It calls a tool that SSHes into the devices and pulls the live BGP summary. Then it interprets the result."
- "This is why the structured data from Labs 1 through 6 matters so much. AI agents work best when they have structured, machine-readable data to work with. A well-organized data model and a validated pipeline give the agent exactly that."

SEGMENT 3: Local Models, No Cloud Required (6:00 - 9:00)

What to show: Show Ollama and the available models.

```
ollama list
```

Talking points:
- "Everything runs locally. No cloud API, no subscription, no data leaving your network."
- "I am using Ollama to run open-source models. Right now I have models ranging from 8 billion parameters up to models with hundreds of billions."
- Show the model list: "Each model has different trade-offs. The smaller models are fast and run on modest hardware. The larger models are more capable but need more RAM."
- "For network operations questions, the mid-range models are surprisingly good. They can interpret BGP session output, analyze OSPF neighbor states, and correlate data across devices."
- "This matters for production use. If you are in a regulated environment where data cannot leave your network, a local model is the only option. And it works."

SEGMENT 4: Live Demo - Fabric Health Query (9:00 - 14:00)

What to show: Ask the AI about fabric health.

```
uv run python -m agent.assistant --backend ollama fabric-qa "Is the fabric healthy? Give a detailed analysis."
```

Talking points:
- "Let me ask the AI a simple question: is the fabric healthy?"
- Show the model picker if applicable. Pick a capable model.
- Let the response stream in.
- "Watch what it is doing. It called the fabric status tool, which connected to all six devices and pulled OSPF neighbor tables, BGP session states, and route counts."

- "Now it is interpreting the results. It is telling us that all OSPF adjacencies are Full, all BGP sessions are Established, route counts are consistent across devices."
- "This is not a canned response. If I had broken something before asking this question, the AI would report the failure and suggest what to investigate."

SEGMENT 5: Live Demo - Specific Questions (14:00 - 19:00)

What to show: Ask 2-3 more specific questions.

Question 1: Route Reflectors

```
uv run python -m agent.assistant --backend ollama fabric-qa "Which devices are route reflectors and how many clients does each one have?"
```

- "It pulled the BGP data and identified both spines as route reflectors, each with four clients: two leafs and two borders. That matches the data model exactly."

Question 2: Reachability

```
uv run python -m agent.assistant --backend ollama fabric-qa "Can leaf1 reach border2? Show me the path."
```

- "It ran a traceroute or checked the routing table and identified the path through the spines. Real routing data, not a guess."

Question 3: What-If Scenario

```
uv run python -m agent.assistant --backend ollama fabric-qa "What would happen if spine1 went down?"
```

- "This is where it gets interesting. The AI reasons about the topology. It knows spine2 is the second route reflector. It knows traffic would reroute through spine2. It can identify which sessions would drop and which would survive."
- "It is not simulating anything. It is reasoning about the topology it learned from the data model and the live state."

SEGMENT 6: Live Demo - Break and Diagnose (19:00 - 23:00)

What to show: Break something, then ask the AI to diagnose it.

Step 1: Break something

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "shutdown"
```

- "I just shut down spine1's link to leaf1. Let me ask the AI what is going on."

Step 2: Ask the AI

```
uv run python -m agent.assistant --backend ollama fabric-qa "Something seems wrong with leaf1.
```

```
Can you check its OSPF and BGP status?"
```

Talking points:
- "Watch it work. It connected to leaf1, pulled the OSPF neighbors, and found that the adjacency with spine1 is down."
- "It is correlating: spine1's interface is down, which means the OSPF adjacency dropped, but the BGP session might still be up through spine2 because of the iBGP full mesh."
- "It is giving me a diagnosis and a suggested action. That is the kind of analysis that takes a network engineer several CLI commands and mental correlation. The AI did it in one question."

Step 3: Fix it

```
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "no shutdown"
```

SEGMENT 7: Why This Works (23:00 - 25:00)

What to show: Back to camera or a summary slide.

Talking points:
- "The reason this works is not because the AI is magic. It is because we spent six labs building structured, validated, machine-readable infrastructure."
- "The data model gives the AI context about the intended design. The validation framework gives it confidence that the intent is correct. The live device access gives it real operational data. The tests give it a definition of 'healthy.'"
- "Without that foundation, the AI would be guessing. With it, the AI is reasoning about real data within a well-defined framework."
- "This is the payoff of Network as Code. Not just automation, not just CI/CD, but a network that is structured enough for both humans and AI to understand."

SEGMENT 8: The Series Close (25:00 - 27:00)

What to show: Series overview showing all seven labs.

Talking points:
- "That is Lab 7, and that is the series. Seven labs, each building on the last."
- "Lab 1: a data model that describes your network's intent. Lab 2: four layers of validation. Lab 3: automated config generation. Lab 4: a CI/CD pipeline. Lab 5: post-change testing. Lab 6: drift detection and reconciliation. Lab 7: AI integration."
- "Together, they form a complete Network as Code stack. From YAML to running routers, validated, tested, and monitored."
- "If you want to build this yourself, the full lab package is at [your link]. All seven build guides, the complete repository, the visual diagrams, and the PDF guides. $49."
- "If you have been following along with just the YouTube videos, I hope you have a clear picture of what Network as Code looks like in practice and why it matters."

- "Thank you for watching the series. If there is enough interest, I will do a follow-up series on scaling this to larger fabrics, integrating with NetBox, and building custom MCP tools."
- "Subscribe, and I will see you in the next one."

Commands Reference (Quick Copy)

```
# List available models
ollama list

# AI fabric health query
uv run python -m agent.assistant --backend ollama fabric-qa "Is the fabric healthy? Give a detailed analysis."

# AI route reflector query
uv run python -m agent.assistant --backend ollama fabric-qa "Which devices are route reflectors?"

# AI reachability query
uv run python -m agent.assistant --backend ollama fabric-qa "Can leaf1 reach border2?"

# AI diagnostic query
uv run python -m agent.assistant --backend ollama fabric-qa "Something seems wrong with leaf1. Check its status."

# Break something for demo
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "shutdown"

# Fix it
docker exec clab-nac-spine-leaf-spine1 vtysh -c "configure terminal" -c "interface eth1" -c "no shutdown"
```

Do NOT Show On Camera

- The MCP server source code (paid content)
- The agent assistant internals (paid content)
- Tool registration and function definitions (paid content)
- The step-by-step integration process (paid content)
- Prompt engineering details (paid content)

DO Show On Camera

- Ollama model list
- AI responses to live queries (the output, not the code that produces it)
- The AI diagnosing a real issue with live data
- The model picker (if applicable)
- The concept: structured data + tools = reliable AI

- Before/after of breaking something and the AI catching it